

5

数组和广义表

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 数组
- 矩阵
 - 特殊矩阵
 - 稀疏矩阵
- 广义表

目录

	1	数组
	1.1	数组的定义
	1.2	数组的顺序表示和实现
	2	矩阵
	3	广义表

数组

- 数组是一种数据结构，一般作为程序设计语言的固有类型。
- 数组可以看成是线性表的推广。
- 科学计算中涉及到大量的矩阵问题，被描述成一个二维数组。
 - 当矩阵规模很大且具有特殊结构（ 对角矩阵、三角矩阵、对称矩阵、稀疏矩阵等 ），为减少程序的时间和空间需求，采用自定义的描述方式。

基本概念

- 数组是一组偶对（下标值，数据元素值）的集合。
- 在数组中，对于一组有意义的下标，都存在一个与其对应的值。
- 一维数组对应着一个下标值，二维数组对应着两个下标值，如此类推。
- 数组是由 n ($n > 1$) 个具有相同数据类型的数据元素 $a_1, a_2, \dots, a_n$ 组成的有序序列，且该序列必须存储在一块地址连续的存储单元中。

基本概念

- 特点

- 数组中的数据元素具有相同数据类型。
- 数组是一种随机存取结构，给定一组下标，就可以访问与其对应的数据元素。
- 数组中的数据元素个数是固定的。

数组的抽象数据类型定义

- 定义

ADT Array {

数据对象： $D = \{ a_{j_1, j_2, \dots, j_n} \mid a_{j_1, j_2, \dots, j_n} \in \text{ElemSet}, \text{数组元素第 } i \text{ 维下标 } j_i = 0, 1, \dots, b_i - 1, b_i \text{ 是数组第 } i \text{ 维的长度}, i = 1, 2, \dots, n, n > 0 \text{ 称为数组的维数} \}$

数据关系： $R = \{ R_1, R_2, \dots, R_n \}$

$R_i = \{ \langle a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_i + 1, \dots, j_n} \rangle \mid 0 \leq j_k \leq b_k - 1, 1 \leq k \leq n \text{ 且 } k \neq i, 0 \leq j_i \leq b_i - 2, \}$

$a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_i + 1, \dots, j_n} \in D \}$

基本操作：.....

} ADT Array

数组的抽象数据类型定义

- 由上述定义知， n 维数组中有 $b_1 \times b_2 \times \dots \times b_n$ 个数据元素，每个数据元素都受到 n 维关系的约束。

• 直观的 n 维数组

- 以二维数组为例讨论。将二维数组看成是一个定长的线性表，其每个元素又是一个定长的线性表。
- 设二维数组 $A = (a_{ij})_{m \times n}$ ，则 $A = (\alpha_1, \alpha_2, \dots, \alpha_p)$ ， $p = m$ 或 n
- 其中每个数据元素 α_j 是一个列向量(线性表)：
 - $\alpha_j = (\alpha_{1,j}, \alpha_{2,j}, \dots, \alpha_{m,j}) \quad 1 \leq j \leq n$
 - 或是一个行向量： $\alpha_i = (\alpha_{i,1}, \alpha_{i,2}, \dots, \alpha_{i,n}) \quad 1 \leq i \leq m$
- 如图5-1所示。

数组的抽象数据类型定义

• 二维数组图例形式

$$\mathbf{A} = \begin{pmatrix} \boxed{a_{11}} & \boxed{a_{12}} & \boxed{\dots} & \boxed{a_{1n}} \\ \boxed{a_{21}} & \boxed{a_{22}} & \boxed{\dots} & \boxed{a_{2n}} \\ \boxed{\vdots} & \boxed{\vdots} & \boxed{\ddots} & \boxed{\vdots} \\ \boxed{a_{m1}} & \boxed{a_{m2}} & \boxed{\dots} & \boxed{a_{mn}} \end{pmatrix} \quad \mathbf{A} = \begin{pmatrix} \left(\boxed{a_{11}} & \boxed{a_{12}} & \boxed{\dots} & \boxed{a_{1n}} \right) \\ \left(\boxed{a_{21}} & \boxed{a_{22}} & \boxed{\dots} & \boxed{a_{2n}} \right) \\ \left(\boxed{\vdots} & \boxed{\vdots} & \boxed{\ddots} & \boxed{\vdots} \right) \\ \left(\boxed{a_{m1}} & \boxed{a_{m2}} & \boxed{\dots} & \boxed{a_{mn}} \right) \end{pmatrix}$$

(a) 矩阵表示形式

(b) 行向量的一维数组形式

$$\mathbf{A} = \begin{pmatrix} \boxed{a_{11}} & \boxed{a_{12}} & \boxed{\dots} & \boxed{a_{1n}} \\ \boxed{a_{21}} & \boxed{a_{22}} & \boxed{\dots} & \boxed{a_{2n}} \\ \boxed{\vdots} & \boxed{\vdots} & \boxed{\ddots} & \boxed{\vdots} \\ \boxed{a_{m1}} & \boxed{a_{m2}} & \boxed{\dots} & \boxed{a_{mn}} \end{pmatrix}$$

(c) 列向量的一维数组形式

目录

	1	数组
	1.1	数组的定义
	1.2	数组的顺序表示和实现
	2	矩阵
	3	广义表

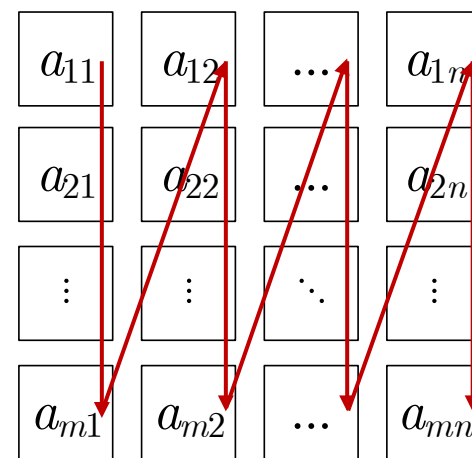
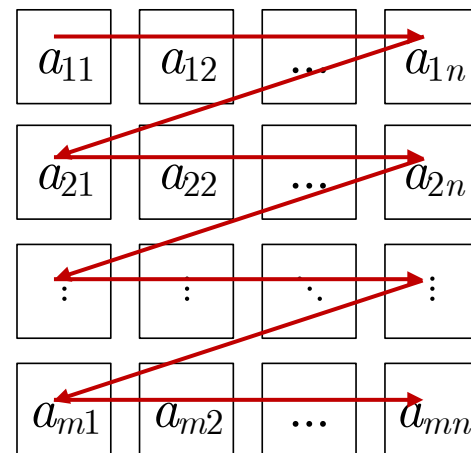
数组的顺序表示和实现

- 数组一般不做插入和删除操作，可采用顺序存储的方法来表示数组。
- 问题：计算机的内存结构是一维(线性)地址结构，对于多维数组，将其存放(映射)到内存一维结构时，有个次序约定问题。即必须按某种次序将数组元素排成一系列序列，然后将这个线性序列存放内存中。
- 二维数组是最简单的多维数组，以此为例说明多维数组存放(映射)到内存一维结构时的次序约定问题。

数组的顺序表示和实现

- 通常有两种顺序存储方式

- 行优先顺序(Row Major Order) : 将数组元素按行排列, 第 $i+1$ 个行向量紧接在第 i 个行向量后面。
- PASCAL、C是按行优先顺序存储的。
- 列优先顺序(Column Major Order) : 将数组元素按列向量排列, 第 $j+1$ 个列向量紧接在第 j 个列向量之后。
- FORTRAN、R、MATLAB是按列优先顺序存储的。

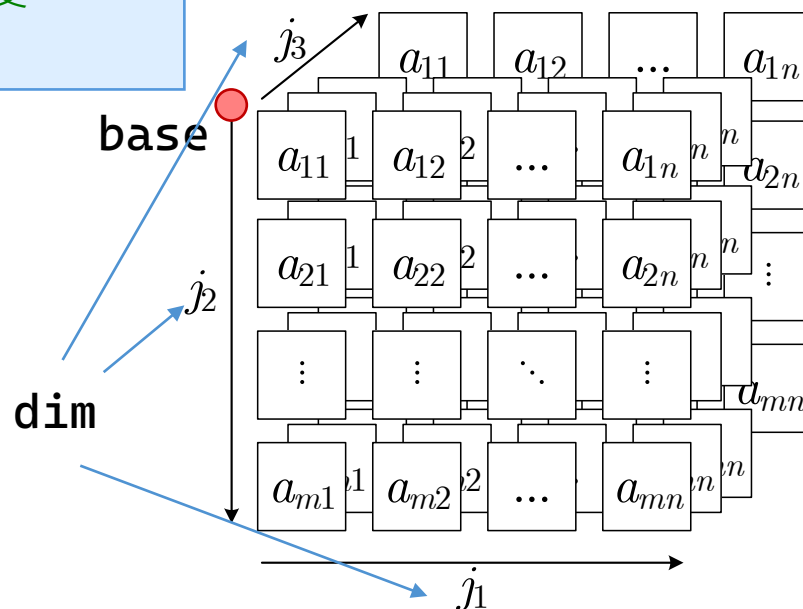


数组的存储结构

• 数组的存储结构

```
typedef struct {  
    int* base;           // 元素基址  
    int dim;             // 维数  
    int* bounds;         // 各维大小 (j)  
    int* constants;      // 该维的元素跨度  
} Array;
```

`constants[i]`指的是第*i*维每前进1步，
相当于最低一层的元素前进几步。
用于定位的时候提高速度避免重复计算
`bounds[i+1]*bounds[i+2]*...
*bounds[dim]`。



数组：初始化

• 初始化

– 设置维度 `A->dim = dim;`

– 设置各维度长度

– 计算元素个数

– 开辟空间

```
A->bounds = (int*)malloc(dim * sizeof(int));  
int elemtotal = 1;  
for (int i = 0; i < dim; ++i) {  
    A->bounds[i] = bounds[i];  
    elemtotal *= bounds[i];  
}
```

```
A->base = (int*)malloc(elemtotal * sizeof(int));  
memset(A->base, 0, elemtotal * sizeof(int));
```

– 计算各维的底层跨度

```
A->constants = (int*)malloc(dim * sizeof(int));  
A->constants[dim - 1] = 1;  
for (int i = dim - 2; i >= 0; --i)  
    A->constants[i] = A->bounds[i + 1] * A->constants[i + 1];
```

```

int InitArray(Array* A, int dim, int bounds[]) {
    if (dim < 1 || dim > MAX_ARRAY_DIM) return ERROR;
    A->dim = dim;
    A->bounds = (int*)malloc(dim * sizeof(int));
    if (!A->bounds) exit(OVERFLOW);
    int elemtotal = 1;
    for (int i = 0; i < dim; ++i) {
        if (bounds[i] <= 0) return UNDERFLOW;
        A->bounds[i] = bounds[i];
        elemtotal *= bounds[i];
    }
    A->base = (int*)malloc(elemtotal * sizeof(int));
    if (!A->base) exit(OVERFLOW);
    memset(A->base, 0, elemtotal * sizeof(int));
    A->constants = (int*)malloc(dim * sizeof(int));
    if (!A->constants) exit(OVERFLOW);
    A->constants[dim - 1] = 1;
    for (int i = dim - 2; i >= 0; --i)
        A->constants[i] = A->bounds[i + 1] * A->constants[i + 1];
    return OK;
}

```

数组：销毁数组

- 销毁：释放所有堆上的空间

```
int DestroyArray(Array* A) {  
    if (!A->base) return ERROR;  
    free(A->base); A->base = NULL;  
    free(A->bounds); A->bounds = NULL;  
    free(A->constants); A->constants = NULL;  
    return OK;  
}
```


数组：定位

- 定位：计算偏移量

$$I = \sum_{i=0}^d c_i \cdot i_i$$

```
int Locate(Array A, int indices[], int* off) {  
    *off = 0;  
    for (int i = 0; i < A.dim; ++i) {  
        if (indices[i] < 0 || indices[i] >= A.bounds[i])  
            return OVERFLOW;  
        *off += A.constants[i] * indices[i];  
    }  
    return OK;  
}
```

数组：赋值与取值

- 赋值与取值：按偏移量进行
$$I = \sum_{i=0}^d c_i \cdot i_i$$

```
int Value(Array A, int* e, int indices[]) {  
    int off;  
    int s = Locate(A, indices, &off);  
    if (s != OK) return s;  
    *e = *(A.base + off);  
    return OK;  
}
```

```
int Assign(Array* A, int e, int indices[]) {  
    int off;  
    int s = Locate(*A, indices, &off);  
    if (s != OK) return s;  
    *(A->base + off) = e;  
    return OK;  
}
```

数组：显示

- 数组的显示是一个递归问题
 - 一维打印一行
 - 二维打印行列，显示行号
 - 三维以上带层次，为分隔的 $N-1$ 维问题

```

void print_array_rec(Array A, int cur_dim, int* indices, int depth) {
    if (cur_dim == A.dim - 1) {
        for (int i = 0; i < A.bounds[cur_dim]; ++i) {
            indices[cur_dim] = i;
            int val;
            Value(A, &val, indices);
            printf("%4d ", val);
        }
        printf("\n");
    } else if (cur_dim == A.dim - 2) {
        for (int i = 0; i < A.bounds[cur_dim]; ++i) {
            indices[cur_dim] = i;
            for (int d = 0; d < depth; ++d) printf(" ");
            printf("行%d: ", i);
            print_array_rec(A, cur_dim+1, indices, depth+1);
        }
    } else {
        for (int i = 0; i < A.bounds[cur_dim]; ++i) {
            indices[cur_dim] = i;
            for (int d = 0; d < depth; ++d) printf(" "); // 缩进
            printf("维%d[%d]:\n", cur_dim, i);
            print_array_rec(A, cur_dim + 1, indices, depth + 1);
        }
    }
}

```

```

void DisplayArray(Array A) {
    if (A.base == NULL) {
        printf("数组未初始化。\\n"); return;
    }
    int total = 1;
    for (int i = 0; i < A.dim; ++i) total *= A.bounds[i];
    printf("数组维度: %d, 总元素数: %d\\n", A.dim, total);
    if (total == 0) return;
    int* indices = (int*)malloc(A.dim * sizeof(int));
    if (!indices)
        printf("内存不足, 无法显示。\\n"); return;
    if (A.dim == 1) { // 一维: 直接打印一行
        printf("一维数组: \\n");
        for (int i = 0; i < A.bounds[0]; ++i) {
            indices[0] = i;
            int val;
            Value(A, &val, indices);
            printf("%4d ", val);
        }
        printf("\\n");
    } else if (A.dim == 2) { // 二维: 打印矩阵, 每行显示行号

```

```

printf("数组 (%d行 x %d列) : \n", A.bounds[0], A.bounds[1]);
for (int i = 0; i < A.bounds[0]; ++i) {
    indices[0] = i;
    printf("行%d: ", i);
    for (int j = 0; j < A.bounds[1]; ++j) {
        indices[1] = j;
        int val;
        Value(A, &val, indices);
        printf("%4d ", val);
    }
    printf("\n");
}
} else { // 三维及以上: 递归打印带层次结构
    printf("多维数组 (各维大小: ");
    for (int i = 0; i < A.dim; ++i) printf("%d%s",
A.bounds[i], (i == A.dim - 1 ? ") : \n" : " x "));
    print_array_rec(A, 0, indices, 0);
}
free(indices);
}

```

目录

	1	数组
	2	矩阵的压缩存储
	2.1	特殊矩阵
	2.2	稀疏矩阵
	3	广义表

矩阵的压缩存储

- 矩阵是一种常用的数学对象，在高级语言编程时，通常将一个矩阵描述为一个二维数组。
- 对于高阶矩阵，若其中非零元素呈某种规律分布或者矩阵中有大量的零元素，若仍然用常规方法存储，可能存储重复的非零元素或零元素，将造成存储空间的大量浪费。
- 对这类矩阵进行压缩存储：
 - 多个相同的非零元素只分配一个存储空间；
 - 零元素不分配空间。

对称矩阵

- 特殊矩阵：是指非零元素或零元素的分布有一定规律的矩阵。
- 对称矩阵：若一个 n 阶方阵 $A=(a_{ij})_{n \times n}$ 中的元素满足性质： $a_{ij}=a_{ji}$, $1 \leq i, j \leq n$ 且 $i \neq j$ ，则称 A 为对称矩阵。
 - 对称矩阵中的元素关于主对角线对称，因此，让每一对对称元素 a_{ij} 和 a_{ji} ($i \neq j$) 分配一个存储空间，则 n^2 个元素压缩存储到 $n(n+1)/2$ 个存储空间，能节约近一半的存储空间。

$$A = \begin{pmatrix} 5 & 0 & 8 & 0 & 0 \\ 1 & 8 & 9 & 2 & 6 \\ 3 & 0 & 2 & 5 & 1 \\ 7 & 0 & 6 & 1 & 3 \end{pmatrix}$$

对称矩阵

- 对称矩阵

- 存储顺序（行优先顺序为例）：

- $a_{11} \ a_{21} \ a_{22} \ a_{31} \ a_{32} \ a_{33} \ \dots \ a_{n1} \ a_{n2} \ \dots \ a_{nn}$

$$k = \begin{cases} \frac{i(i-1)}{2} + (j-1), & i \geq j \\ \frac{j(j-1)}{2} + (i-1), & i < j \end{cases} \quad i = \left\lfloor \frac{1 + \sqrt{1 + 8k}}{2} \right\rfloor, j = k - \frac{i(i-1)}{2} + 1$$

三角矩阵

- 三角矩阵

- 以主对角线划分，三角矩阵有上三角和下三角两种。
- 上三角矩阵的下三角（不包括主对角线）中的元素均为常数 c （一般为0）。下三角矩阵正好相反，它的主对角线上方均为常数。
- 三角矩阵中的重复元素 c 可共享一个存储空间，其余的元素正好有 $n(n+1)/2$ 个。三角矩阵存储到向量 $[0, n(n+1)/2)$ 中，其中 c 存放在向量的第 $n(n+1)/2$ 个分量中。
- 下三角的转换关系参考前页对称矩阵。上三角矩阵的情况将坐标对调。

对角矩阵

- 对角矩阵：矩阵中，除了主对角线和主对角线上或下方若干条对角线上的元素之外，其余元素皆为零。
 - 所有的非零元素集中在以主对角线为中心的带状区域中。

$$A = \begin{pmatrix} a_{11} & a_{12} & 0 & \dots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \dots & 0 \\ 0 & a_{32} & a_{33} & a_{34} & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & a_{n-1 \ n-2} & a_{n-1 \ n-1} & a_{n-1 \ n} \\ 0 & \dots & 0 & 0 & a_{n \ n-1} & a_{n \ n} \end{pmatrix}$$

对角矩阵

- 对角矩阵可按行优先顺序或对角线顺序，将其压缩存储到一个向量中，并且也能找到每个非零元素和向量下标的对应关系。
- 矩阵行列和偏移量的对应关系无需背诵，根据实际需要自行计算。

目录

	1	数组
	2	矩阵的压缩存储
	2.1	特殊矩阵
	2.2	稀疏矩阵
	3	广义表

稀疏矩阵

- 稀疏矩阵(Sparse Matrix)：没有确切的定义。
 - 稀疏因子：设矩阵A是一个 $n \times m$ 的矩阵中有 s 个非零元素，设 $\delta = s / (n \times m)$ ，称 δ 为稀疏因子。
 - 如果某一矩阵的稀疏因子 δ 满足 $\delta \leq 0.05$ 时称为稀疏矩阵。
- 压缩存储：只存储非0元素，行、列、元素值。
 - 一个三元组 (i, j, a_{ij}) 唯一确定稀疏矩阵的一个非零元素。

$$\mathbf{A} = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \\ 0 & 0 & 24 & 0 & 0 & 2 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -7 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 & 0 \end{pmatrix}$$



$((1,2,12), (1,3,9), (3,1,-3), (3,8,4),$
 $(4,3,24), (5,2,18), (6,7,-7), (7,4,-6))$

三元组顺序表

- 若以行序为主序，稀疏矩阵中所有非0元素的三元组，就可以得构成该稀疏矩阵的一个三元组顺序表。
- 三元组顺序表定义

```
typedef struct {  
    int row;  
    int col;  
    int e;  
} Triple;  
typedef struct {  
    Triple data[MAXSIZE + 1]; // 三元组表，从下标1开始存储  
    int m, n;                 // 行数、列数  
    int len;                   // 非零元个数  
} SparseMatrix;
```

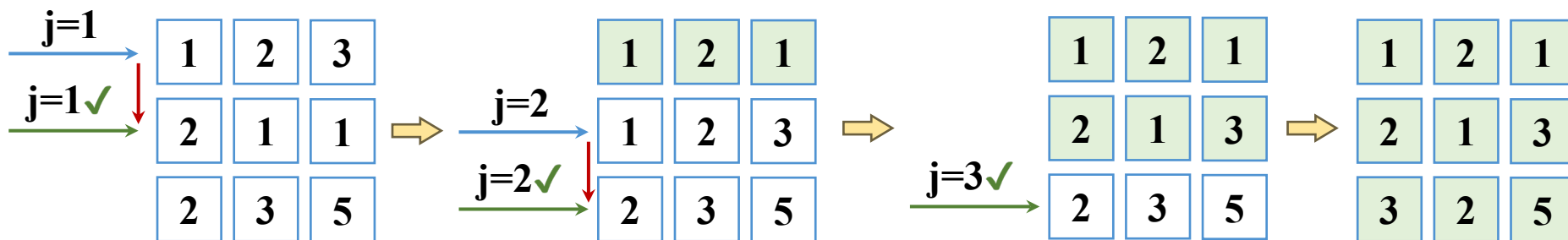

稀疏矩阵的转置（简单）

- 主要操作：行、列互换；问题：换完保持行主序排列

0	3	0
1	0	5
0	0	0



0	1	0
3	0	0
0	5	0



- 时间复杂度： $O(n \times L)$ 行乘以长

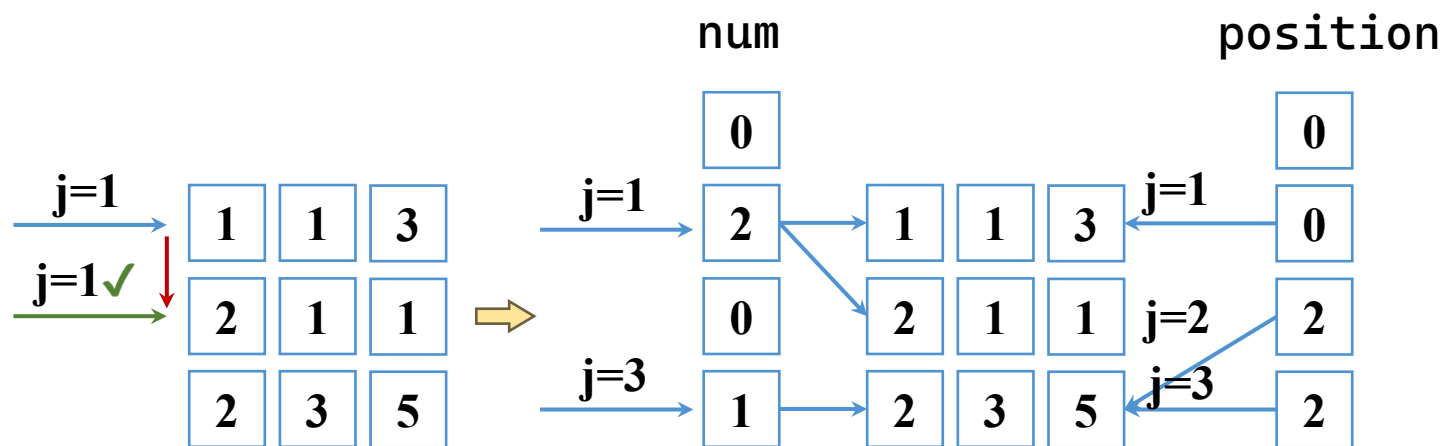
```

int transpose(const SparseMatrix* A, SparseMatrix* B) {
    if (A == NULL || B == NULL) return ERROR;
    B->m = A->n; B->n = A->m; B->len = A->len;
    if (B->len == 0) return OK;
    int j = 1;    // B 中三元组当前存放位置
    for (int col = 1; col <= A->n; col++) {
        for (int i = 1; i <= A->len; i++) {
            if (A->data[i].col == col) {
                B->data[j].row = A->data[i].col;
                B->data[j].col = A->data[i].row;
                B->data[j].e = A->data[i].e;
                j++;
            }
        }
    }
    return OK;
}

```

稀疏矩阵的转置（快速）

- 简单转置的改进思路：只需要一层遍历
 - 算出每一列在目标三元组数组的偏移量
 - 每次处理一个数组，偏移量自增一
- 时间复杂度： $O(n + L)$ 行加长



第j列存储，从
position[j]开始

```

int fast_transpose(const SparseMatrix* A, SparseMatrix* B) {
    if (A == NULL || B == NULL) return ERROR;
    B->m = A->n; B->n = A->m; B->len = A->len;
    if (B->len == 0) return OK;
    int num[MAXROW + 2] = { 0 };
    int position[MAXROW + 2];
    for (int t = 1; t <= A->len; t++) // 统计每列非零元个数
        num[A->data[t].col]++;
    position[1] = 1; // 计算每列第一个元素在 B 中的起始位置
    for (int col = 2; col <= A->n; col++)
        position[col] = position[col - 1] + num[col - 1];
    for (int p = 1; p <= A->len; p++) { // 转置
        int col = A->data[p].col;
        int q = position[col];
        B->data[q].row = A->data[p].col;
        B->data[q].col = A->data[p].row;
        B->data[q].e = A->data[p].e;
        position[col]++;
    }
    return OK;
}

```

应用：矩阵乘法

- 于 A 的第 i 行与 B 的第 j 列对应元素乘积之和

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}, \quad 1 \leq i \leq m, 1 \leq j \leq p$$

- 方法

– 遍历每一行 i

```
for (row=1; row<=Q->m; row++) {
```

– 遍历该行存储的每一列 k

```
for (p=1; p<=M->len; p++) {  
    if (M->data[p].row!=row) continue;
```

– 对应计算 $a_{ik}b_{kj}$

```
for (q=1; q<=N->len; q++) {  
    if (N->data[q].row != M->data[p].col) continue;
```

– 并相加存入 c_{ij}

```
temp[N->data[q].col]+=M->data[p].e*N->data[q].e;
```

```
for (int col = 1; col <= Q->n; col++)  
    if (temp[col] != 0) {  
        Q->len++; Q->data[Q->len].row=row;  
        Q->data[Q->len].col=col; Q->data[Q->len].e=temp[col];  
    }
```

应用：矩阵乘法

- 改进：记录每一行从data的偏移量到first

```
void build_first(SparseMatrix* A) {  
    int num[MAXROW + 2] = { 0 };  
    for (int i = 1; i <= A->len; i++) {  
        num[A->data[i].row]++;  
    }  
    A->first[1] = 1;  
    for (int i = 2; i <= A->m; i++) {  
        A->first[i] = A->first[i - 1] + num[i - 1];  
    }  
    A->first[A->m + 1] = A->len + 1;  
}
```

```

int multiply_matrix(const SparseMatrix* M,
                   const SparseMatrix* N, SparseMatrix* Q) {
    if (M == NULL || N == NULL || Q == NULL) return ERROR;
    if (M->n != N->m) return ERROR;
    Q->m = M->m; Q->n = N->n; Q->len = 0;
    if (M->len == 0 || N->len == 0) return OK;    // 结果为零矩阵
    int temp[MAXROW + 2] = { 0 };
    for (int row = 1; row <= Q->m; row++) {
        for (int col = 1; col <= Q->n; col++) temp[col] = 0;
        // 遍历 M 的第 row 行
        for (int p = M->first[row]; p < M->first[row + 1]; p++) {
            int col_m = M->data[p].col, val_m = M->data[p].e;
            // 遍历 N 的第 col_m 行
            for (int q = N->first[col_m]; q < N->first[col_m + 1]; q++) {
                int col_n = N->data[q].col, val_n = N->data[q].e;
                temp[col_n] += val_m * val_n;
            }
        }
    }
}

```

```

// 将非零结果存入 Q
for (int col = 1; col <= Q->n; col++) {
    if (temp[col] != 0) {
        if (Q->len >= MAXSIZE) {
            printf("乘积矩阵非零元数量超过 %d, 部分截断。
\n", MAXSIZE);
            return ERROR;
        }
        Q->len++;
        Q->data[Q->len].row = row;
        Q->data[Q->len].col = col;
        Q->data[Q->len].e = temp[col];
    }
}
return OK;
}

```


十字链表

- 当矩阵的非零元个数和位置在操作过程中变化较大时，就不宜采用顺序存储结构来表示三元组的线性表。
 - 例如，在作“将矩阵B加到矩阵A上”的操作时，由于非零元的插入或删除将会引起A.data中元素的移动。为此，对这种类型的矩阵，采用链式存储结构表示比三元组的线性表更方便。
- 矩阵中非0元素的结点所含的域有：行、列、值、行指针(指向同一行的下一个非0元)、列指针(指向同一列的下一个非0元)。其次，十字交叉链表还有一个头结点，结点的结构如图5-10所示。

十字链表

- 稀疏矩阵中同一行的非0元素的由right指针域链接成一个行链表，由down指针域链接成一个列链表。则每个非0元素既是某个行链表中的一个结点，同时又是某个列链表中的一个结点，所有的非0元素构成一个十字交叉的链表。称为十字链表。用两个一维数组分别存储行链表的头指针和列链表的头指针。
- 对于图5-11(a)的稀疏矩阵A，对应的十字交叉链表如图5-11(b)所示，结点的描述如下：

十字链表

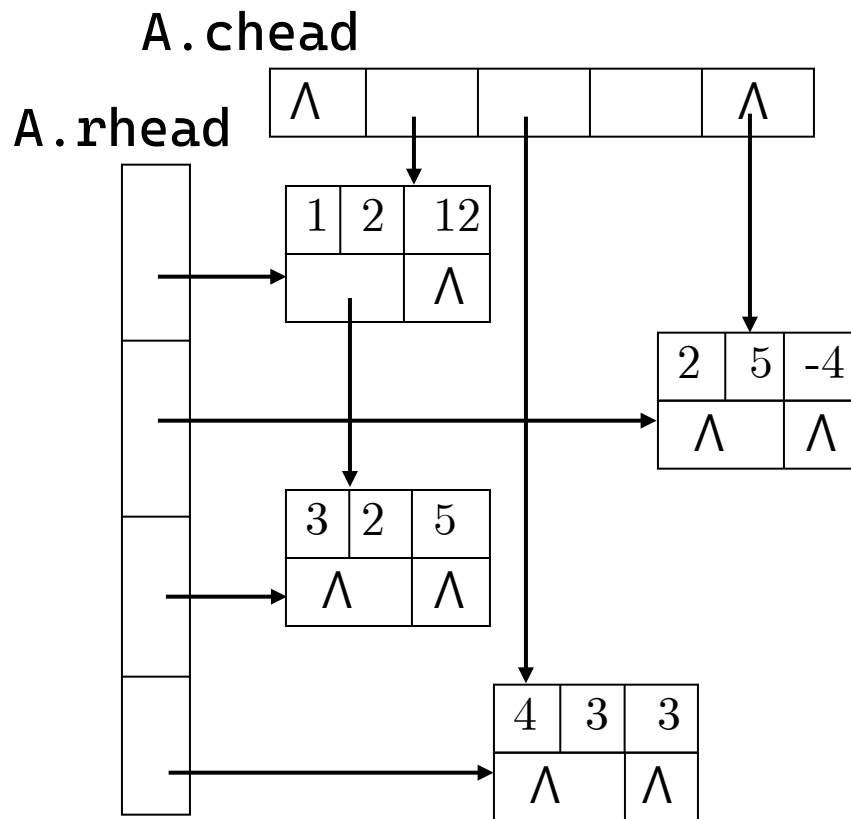
- 十字链表的定义

```
typedef struct OLNode {  
    int row, col, value;  
    struct OLNode* right, * down;  
} OLNode, * OLink;  
  
typedef struct {  
    OLink* rhead, * chead;    // 行、列头指针数组  
    int m, n, len;           // 行数、列数、非零元个数  
} CrossList;
```

十字链表

$$\mathbf{A} = \begin{pmatrix} 0 & 12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -4 \\ 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \end{pmatrix}$$

(a) 稀疏矩阵



(b) 稀疏矩阵的十字交叉链表

图5-11 稀疏矩阵及其十字交叉链表

目录

	1	数组
	2	矩阵的压缩存储
	3	广义表

广义表

- 广义表(Lists，又称为列表)：是由 $n(n \geq 0)$ 个元素组成的有穷序列： $LS = (a_1, a_2, \dots, a_n)$ ，其中 a_i 或者是原子项，或者是一个广义表。
 - 广义表是线性表的推广和扩充，在人工智能领域应用广泛。
 - 线性表定义为 n 个元素 $a_1, a_2, \dots, a_n$ 的有穷序列，序列中的元素是具有相同数据类型的原子项(Atom)。
 - 所谓原子项可以是一个数或一个结构，是指结构上不可再分的。若放松对元素的这种限制，容许它们具有其自身结构，就产生了广义表的概念。

广义表

- 广义表： LS ，长度： n
 - 习惯上：原子用小写字母，子表用大写字母。
 - 广义表中所包含的元素(包括原子和子表)的个数。
- 子表：
 - 若 a_i 是广义表，则称为 LS 的子表。
 - 广义表中括号的最大层数称为表深(度)。
- 表头：
 - 若广义表 LS 非空时 a_1 (表中第一个元素) 称为表头
 - 其余元素组成的子表 $(a_2, a_3, \dots, a_n)$ 称为表尾

广义表实例

- 有关广义表的这些概念的例子如表5-2所示。

表5-2 广义表及其示例

广 义 表	表长n	表深h
$A=()$	0	0
$B=(e)$	1	1
$C=(a,(b,c,d))$	2	2
$D=(A,B,C)$	3	3
$E=(a,E)$	2	∞
$F=(())$	1	2

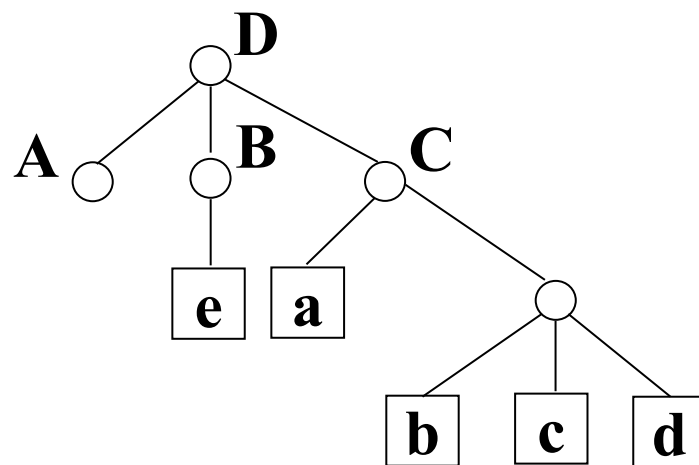


图5-12 广义表的图形表示

广义表的重要结论

- 广义表的重要结论：

- 广义表的元素可以是原子，也可以是子表，子表的元素又可以是子表，...。即广义表是一个多层次的结构。
- 广义表可以被其它广义表所共享，也可以共享其它广义表。广义表共享其它广义表时通过表名引用。
- 广义表本身可以是一个递归表。
- 根据对表头、表尾的定义，任何一个非空广义表的表头可以是原子，也可以是子表，而表尾必定是广义表。

广义表的存储结构

- 由于广义表中的数据元素具有不同的结构，通常用链式存储结构表示，每个数据元素用一个结点表示。因此，广义表中就有两类结点：
 - 一类是表结点，用来表示广义表项，由标志域，表头指针域，表尾指针域组成；
 - 另一类是原子结点，用来表示原子项，由标志域，原子的值域组成。如图5-13所示。

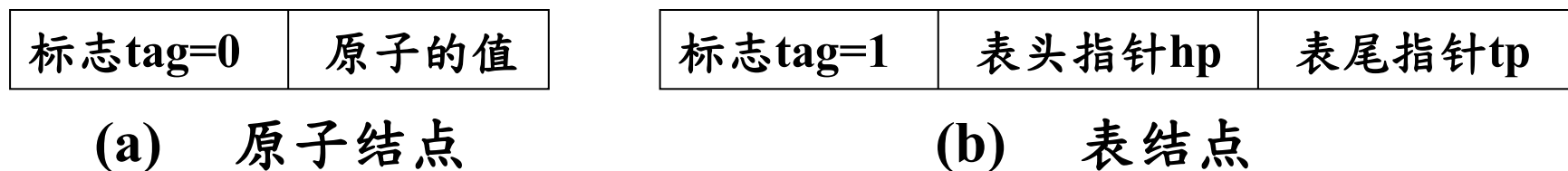


图5-13 广义表的链表结点结构示意图

广义表的定义

- 只要广义表非空，都是由表头和表尾组成。即一个确定的表头和表尾就唯一确定一个广义表。
- 广义表的定义

```
typedef struct GLNode {  
    int tag; // 1-表结点;0-原子结点  
    union {  
        int value; // 原子结点的值域  
        struct {  
            struct GLNode* hp, * tp;  
        } ptr; // ptr和atom两成员共用  
    } Gdata;  
} GLNode; // 广义表结点类型
```

```
typedef struct GLNode {  
    int tag;  
    union {  
        int value;  
        struct GLNode* hp;  
    };  
    int GLNode* tp;  
} GLNode;
```

广义表

- 例：对 $A=()$, $B=(e)$, $C=(a, (b, c, d))$, $D=(A, B, C)$, $E=(a, E)$ 的广义表的存储结构如图5-14所示。

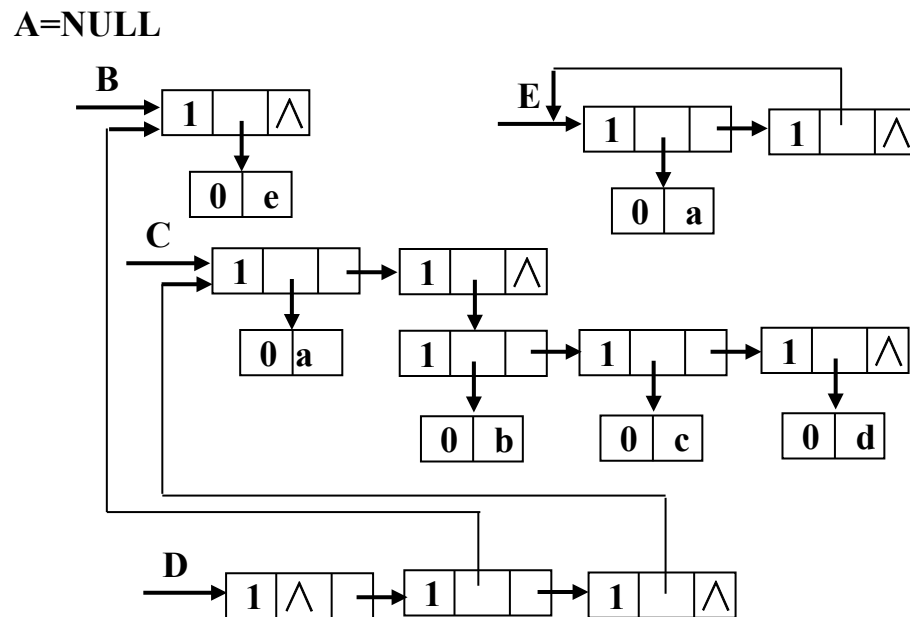
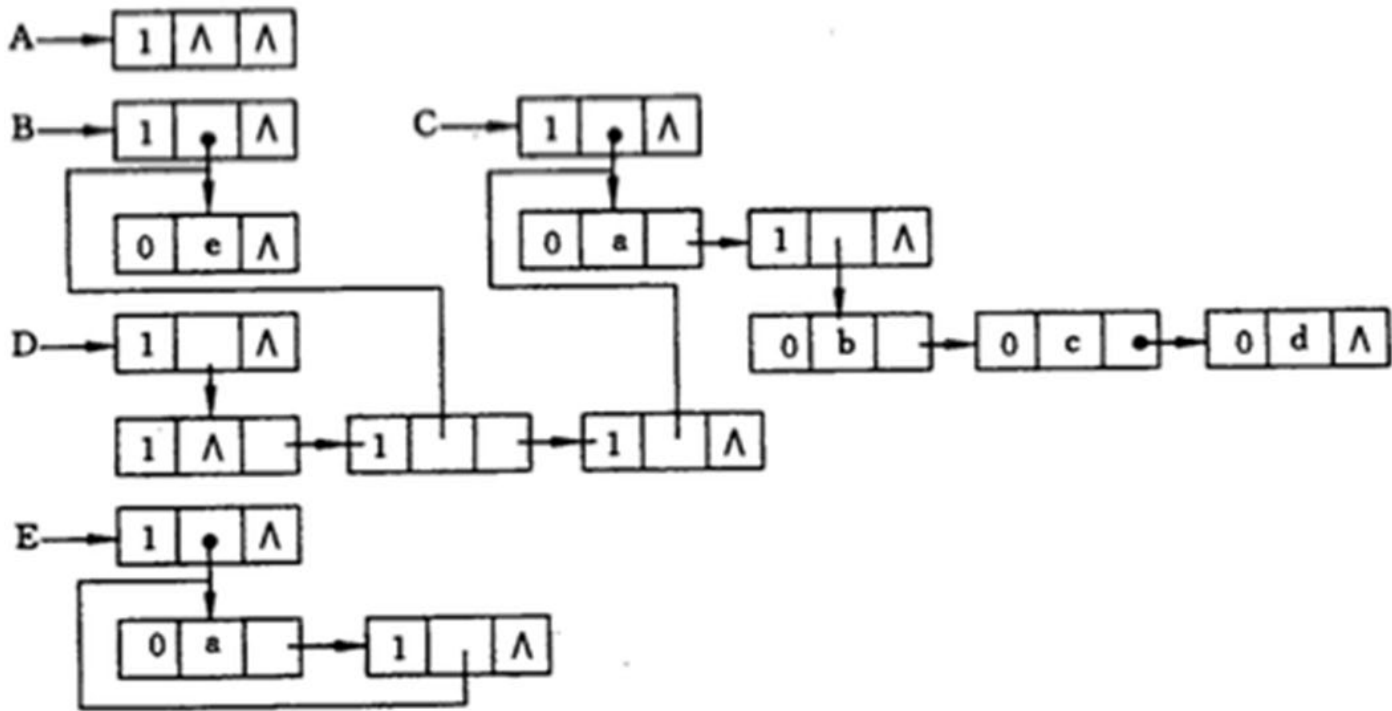


图5-14 广义表的存储结构示意图

广义表



广义表的结构特点

- 广义表中的数据元素有相对次序;
- 广义表的长度定义为最外层包含的元素个数;
- 广义表的深度定义为所包括弧的重数;
 - 注意:“原子”的深度为“0”;“空表”的深度为1
- 表头可以是原子或列表;表尾必定是列表。
- 广义表可以是一个递归的表;
 - 递归表的深度是无穷值,长度是有限值。
- 任何非空广义表 LS 均可分解为表头 $\text{Head}(LS)=a_1$ 和表尾 $\text{Tail}(LS)=(a_2, a_3, \dots, a_n)$ 两部分

广义表的基本运算

- 广义表的基本运算，除包括线性表的基本运算外，还有求深度、取表头、取表尾、遍历等。
- 这些运算中大部分与对应的线性表、树或者图的运算类似，只是取表头和取表尾是广义表特有的运算。

谢谢观看

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang